



KIẾN TRÚC VÀ THIẾT KẾ PHẦN MỀM

Chương 8: Các mẫu thiết kế hành vi

ThS. Nguyễn Văn Hữu Hoàng

I NỘI DUNG

Nội dung của chương này trình bày các mẫu thiết kế:

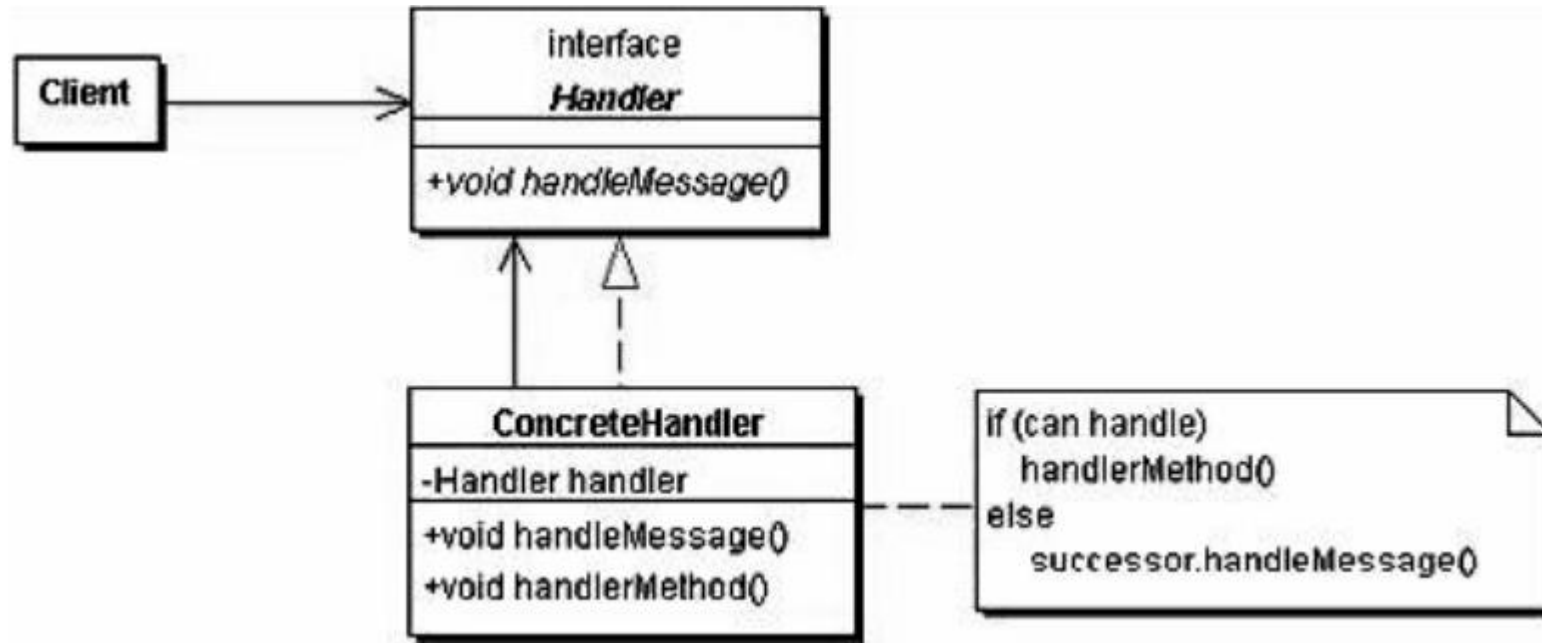
- Mẫu chuỗi trách nhiệm (chain of responsibility)
- Mẫu lệnh (command)
- Mẫu lặp (iterator)
- Mẫu phiên dịch (interpreter)
- Mẫu trung gian (mediator)
- Mẫu hồi tưởng (memento)
- Mẫu quan sát (observer)
- Mẫu chiến lược (strategy)
- Mẫu trạng thái (state)
- Mẫu phương thức mẫu (template method)

| 8.1 MẪU CHUỖI TRÁCH NHIỆM

8.1.1 Đặt vấn đề

- Mẫu chuỗi trách nhiệm (chain of responsibility) thiết lập một chuỗi đối tượng bên trong một hệ thống, trong đó các thông điệp hoặc có thể được thực hiện ở tại một mức nơi mà nó được nhận lần đầu hoặc được chuyển đến một đối tượng để thực thi.

| 8.1.2 CẤU TRÚC MẪU



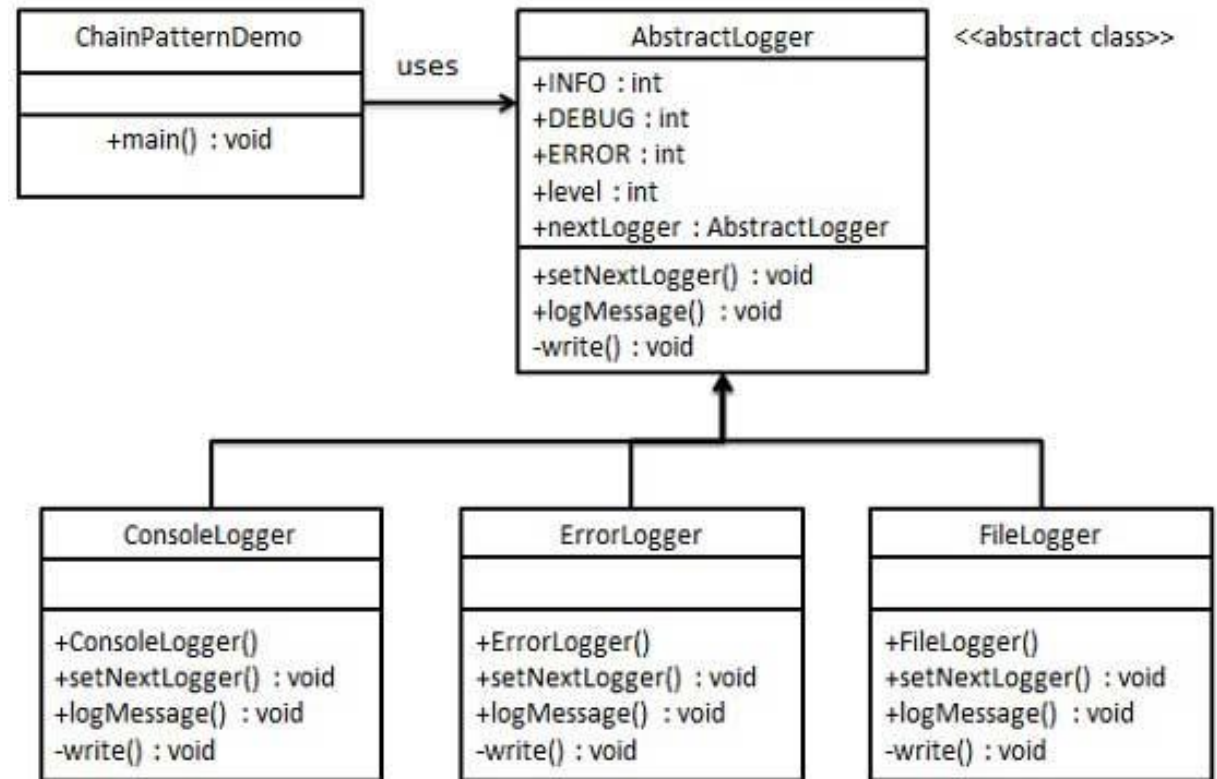
- **Handler**: là một giao tiếp định nghĩa phương thức sử dụng để chuyển thông điệp qua các lần thực hiện tiếp theo.
- **ConcreteHandler**: là một cài đặt của giao tiếp **Handler**. Nó giữ một tham chiếu đến một **Handler** tiếp theo. Việc thực thi phương thức **handleMessage** có thể xác định làm thế nào để thực hiện phương thức và gọi một **handlerMethod**, chuyển tiếp thông điệp đến cho **Handler** tiếp theo hoặc kết hợp cả hai.

| 8.1.3 TÌNH HUỐNG ÁP DỤNG

- Có một nhóm các đối tượng trong hệ thống có thể đáp ứng tất cả các loại thông điệp giống nhau.
- Các thông điệp phải được thực hiện bởi một vài đối tượng trong hệ thống.
- Các thông điệp được thực thi theo mô hình “thực hiện – chuyển tiếp”, một vài sự kiện có thể được thực hiện ở mức mà chúng được nhận hoặc tạo ra, trong khi một số sự kiện khác phải được chuyển tiếp đến một vài đối tượng khác.

| 8.1.4 VÍ DỤ

- Tạo lớp trừu tượng **AbstractLogger** với các mức level của log. Sau đó tạo 3 loại log mở rộng từ **AbstractLogger**: **ConsoleLogger**, **ErrorLogger**, **FileLogger**.
- Mỗi logger kiểm tra level của thông điệp và in ra hoặc không in mà chuyển thông điệp cho logger tiếp theo xử lý.



VÍ DỤ

Bước 1: Tạo lớp logger trừu tượng.

```
public abstract class AbstractLogger {
    public static int INFO = 1;
    public static int DEBUG = 2;
    public static int ERROR = 3;

    protected int level;

    //next element in chain or responsibility
    protected AbstractLogger nextLogger;

    public void setNextLogger(AbstractLogger nextLogger){
        this.nextLogger = nextLogger;
    }

    public void logMessage(int level, String message){
        if(this.level <= level){
            write(message);
        }
        if(nextLogger != null){
            nextLogger.logMessage(level, message);
        }
    }

    abstract protected void write(String message);
}
```

Bước 2: Tạo các lớp mở rộng.

```
public class ConsoleLogger extends AbstractLogger {

    public ConsoleLogger(int level){
        this.level = level;
    }

    @Override
    protected void write(String message) {
        System.out.println("Standard Console::Logger: " + message);
    }
}
```

```
public class ErrorLogger extends AbstractLogger {

    public ErrorLogger(int level){
        this.level = level;
    }

    @Override
    protected void write(String message) {
        System.out.println("Error Console::Logger: " + message);
    }
}
```

```
public class FileLogger extends AbstractLogger {

    public FileLogger(int level){
        this.level = level;
    }

    @Override
    protected void write(String message) {
        System.out.println("File::Logger: " + message);
    }
}
```

I VÍ DỤ

Bước 3: Tạo các loại logger khác nhau, gán level và thiết lập logger tiếp theo. Logger tiếp theo trong mỗi logger chính là một phần của chuỗi.

```
public class ChainPatternDemo {  
  
    private static AbstractLogger getChainOfLoggers(){  
  
        AbstractLogger errorLogger = new ErrorLogger(AbstractLogger.ERROR);  
        AbstractLogger fileLogger = new FileLogger(AbstractLogger.DEBUG);  
        AbstractLogger consoleLogger = new ConsoleLogger(AbstractLogger.INFO);  
  
        errorLogger.setNextLogger(fileLogger);  
        fileLogger.setNextLogger(consoleLogger);  
  
        return errorLogger;  
    }  
  
    public static void main(String[] args) {  
        AbstractLogger loggerChain = getChainOfLoggers();  
  
        loggerChain.logMessage(AbstractLogger.INFO,  
            "This is an information.");  
  
        loggerChain.logMessage(AbstractLogger.DEBUG,  
            "This is an debug level information.");  
  
        loggerChain.logMessage(AbstractLogger.ERROR,  
            "This is an error information.");  
    }  
}
```

Bước 4: Xem kết quả.

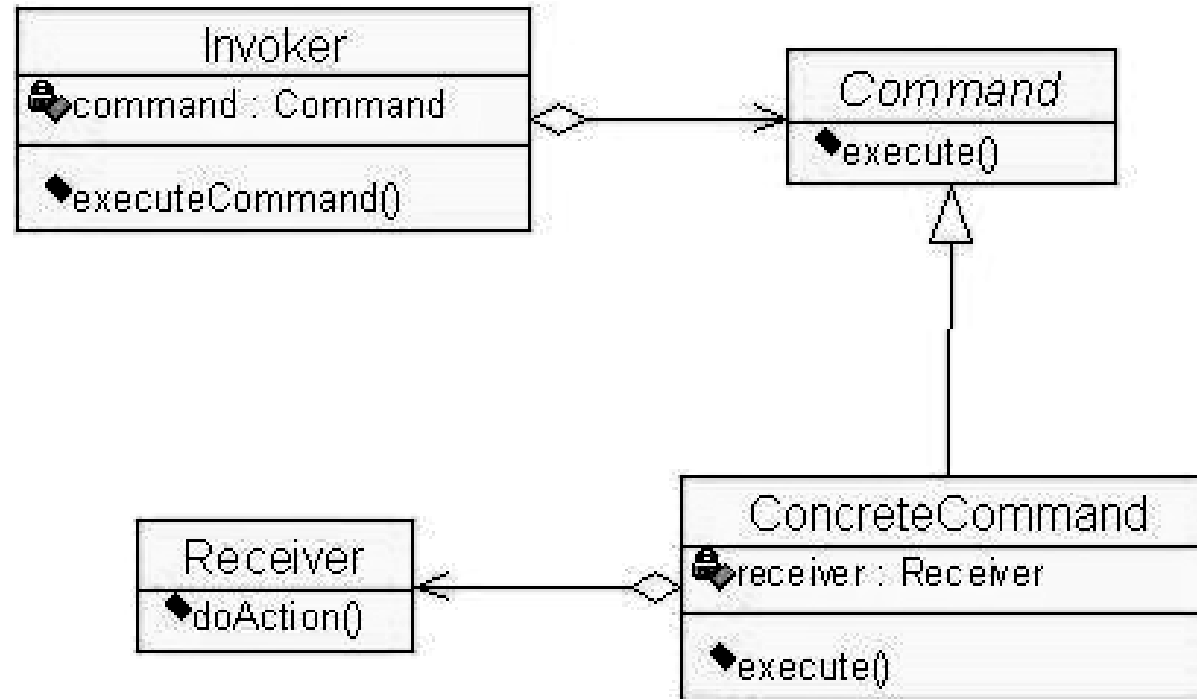
```
Standard Console::Logger: This is an information.  
File::Logger: This is an debug level information.  
Standard Console::Logger: This is an debug level information.  
Error Console::Logger: This is an error information.  
File::Logger: This is an error information.  
Standard Console::Logger: This is an error information.
```

| 8.2 MẪU COMMAND

8.2.1 Đặt vấn đề

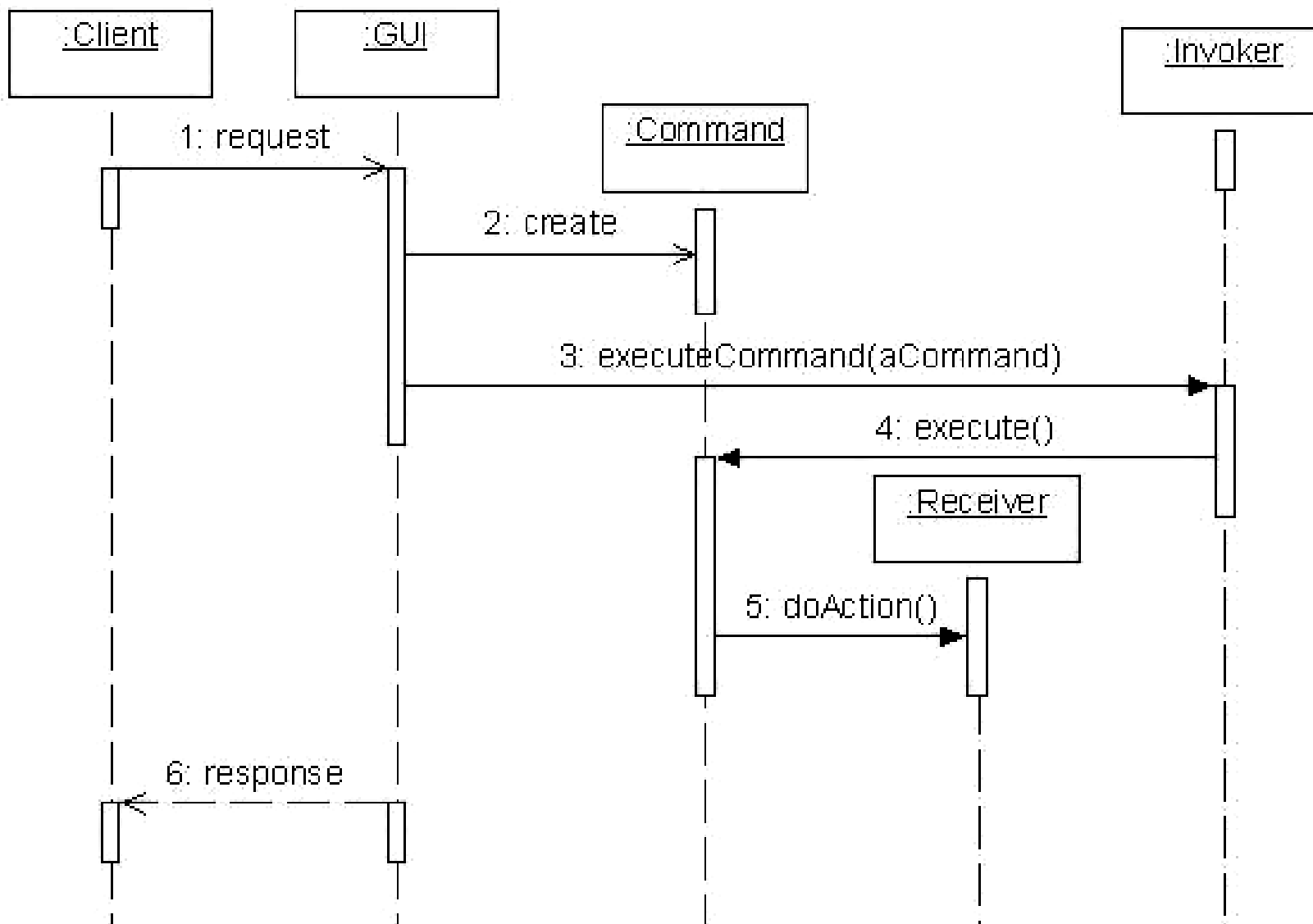
- Đôi khi chúng ta gặp tình huống cần phải gửi yêu cầu đến một đối tượng mà không biết cụ thể về đối tượng nhận yêu cầu cũng như phương thức xử lý yêu cầu. Ví dụ về bộ công cụ giao diện người sử dụng cho phép xây dựng các menu. Rõ ràng, nó không thể xử lý trực tiếp yêu cầu trên các đối tượng menu vì cách thức xử lý phụ thuộc vào từng ứng dụng cụ thể.
- Mẫu **Command** cho phép bộ công cụ trên gửi yêu cầu đến các đối tượng chưa xác định bằng cách biến chính yêu cầu thành một đối tượng.
- Khái niệm quan trọng nhất của mẫu này là lớp trừu tượng **Command** có chức năng định nghĩa giao diện cho việc thực thi các yêu cầu. Trong trường hợp đơn giản nhất, ta chỉ cần định nghĩa một thủ tục ảo **Execute** trên giao diện đó. Các lớp con cụ thể của **Command** sẽ xác định cặp đối tượng nhận yêu cầu - thao tác bằng cách lưu trữ một tham chiếu đến đối tượng nhận yêu cầu và định nghĩa lại thủ tục **Execute** để gọi các thủ tục xử lý.

| 8.2.2 CẤU TRÚC MẪU COMMAND



- **Command**: là một giao tiếp định nghĩa các phương thức cho **Invoker** sử dụng
- **Invoker**: lớp này thực hiện các phương thức của đối tượng **Command**
- **Receiver**: là đích đến của **Command** và là đối tượng thực hiện hoàn tất yêu cầu, nó có tất cả các thông tin cần thiết để thực hiện điều này
- **ConcreteCommand**: là một cài đặt của giao tiếp **Command**. Nó lưu giữ một tham chiếu **Receiver** mong muốn.

| Biểu đồ tuần tự của Command

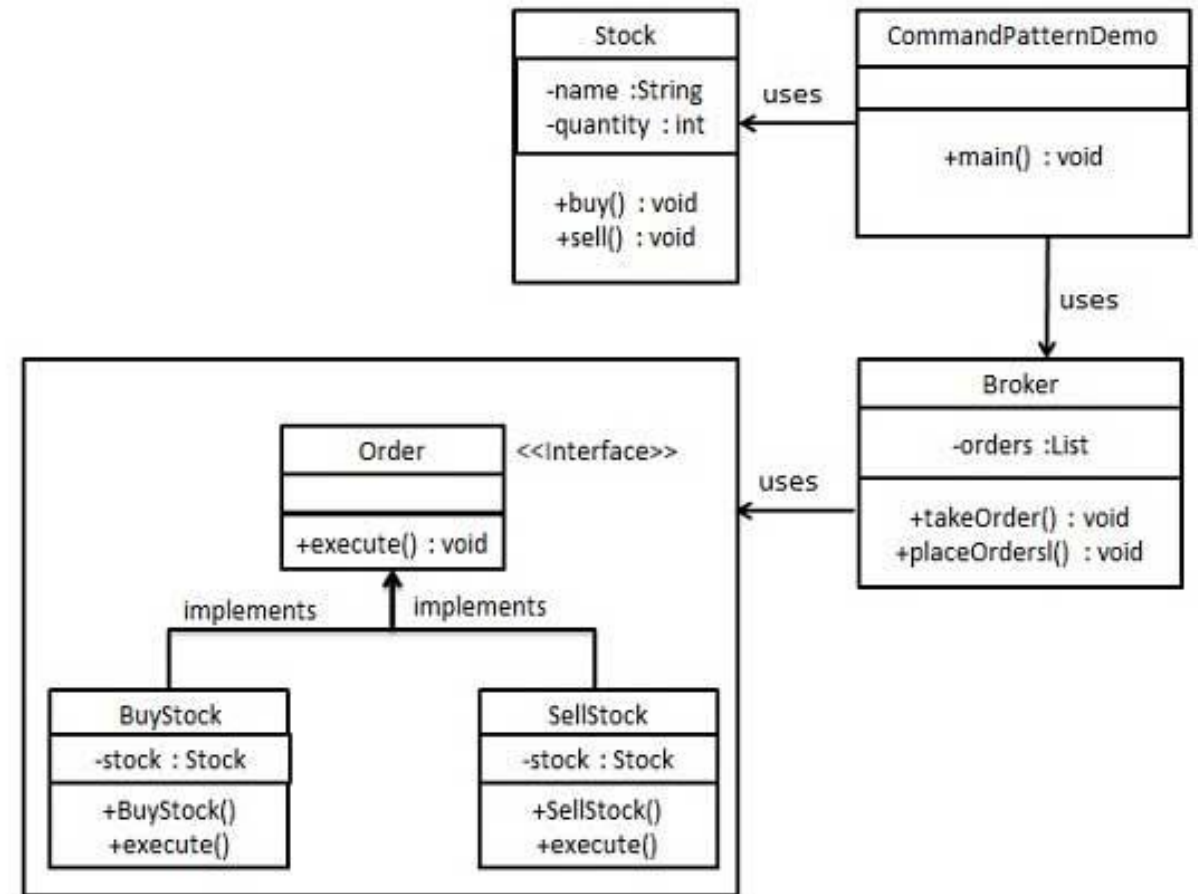


| 8.2.3 TÌNH HUỐNG ÁP DỤNG COMMAND

- Tham số hoá các đối tượng theo thủ tục mà chúng thi hành.
- Xác định, xếp hàng và thực thi các yêu cầu tại những thời điểm khác nhau.
- Cho phép quay lại. Thủ tục `execute` của lớp `Command` có thể lưu lại trạng thái để cho phép quay lại các biến đổi mà nó gây ra. Khi đó lớp `Command` trừu tượng cần định nghĩa thêm hàm `Unexecute` để đảo ngược các biến đổi. Các lệnh đã được thực thi sẽ được lưu trong một danh sách, từ đó cho phép `undo` và `redo` không giới hạn mức.
- Cần hỗ trợ ghi lại các lệnh đã được thực thi để thi hành lại trong trường hợp hệ thống gặp sự cố.

| 8.2.4 VÍ DỤ MẪU THIẾT KẾ COMMAND

- Tạo 1 giao diện **Order** đóng vai trò như là command, tạo lớp **Stock** với vai trò là request.
- Tạo lớp cụ thể **BuyStock** và **SellStock** sử dụng giao diện **Order** thực thi các xử lý command.
- Tạo lớp **Broker** đóng vai trò invoker, nó có thể xử lý các orders.
- Đối tượng **Broker** sử dụng command để xác định đối tượng sẽ thực thi và thực thi với **command** nào dựa trên loại **command**.
- Lớp **CommandPatternDemo** sử dụng **Broker** để demo ứng dụng.



| 8.2.4 VÍ DỤ MẪU THIẾT KẾ COMMAND

Bước 1: Tạo interface Order.

```
public interface Order {  
    void execute();  
}
```

Bước 2: Tạo lớp yêu cầu Stock.

```
public class Stock {  
  
    private String name = "ABC";  
    private int quantity = 10;  
  
    public void buy(){  
        System.out.println("Stock [ Name: "+name+",  
            Quantity: " + quantity + " ] bought");  
    }  
    public void sell(){  
        System.out.println("Stock [ Name: "+name+",  
            Quantity: " + quantity + " ] sold");  
    }  
}
```

Bước 3: Tạo các lớp cụ thể.

```
public class BuyStock implements Order {  
    private Stock abcStock;  
  
    public BuyStock(Stock abcStock){  
        this.abcStock = abcStock;  
    }  
  
    public void execute() {  
        abcStock.buy();  
    }  
}
```

```
public class SellStock implements Order {  
    private Stock abcStock;  
  
    public SellStock(Stock abcStock){  
        this.abcStock = abcStock;  
    }  
  
    public void execute() {  
        abcStock.sell();  
    }  
}
```

| 8.2.4 VÍ DỤ MẪU THIẾT KẾ COMMAND

Bước 4: Tạo lớp invoker.

```
import java.util.ArrayList;
import java.util.List;
public class Broker {
    private List<Order> orderList = new ArrayList<Order>();

    public void takeOrder(Order order){
        orderList.add(order);
    }

    public void placeOrders(){

        for (Order order : orderList) {
            order.execute();
        }
        orderList.clear();
    }
}
```

Bước 5: Sử dụng broker để thực thi command.

```
public class CommandPatternDemo {
    public static void main(String[] args) {
        Stock abcStock = new Stock();

        BuyStock buyStockOrder = new BuyStock(abcStock);
        SellStock sellStockOrder = new SellStock(abcStock);

        Broker broker = new Broker();
        broker.takeOrder(buyStockOrder);
        broker.takeOrder(sellStockOrder);

        broker.placeOrders();
    }
}
```

Bước 6: Xem kết quả.

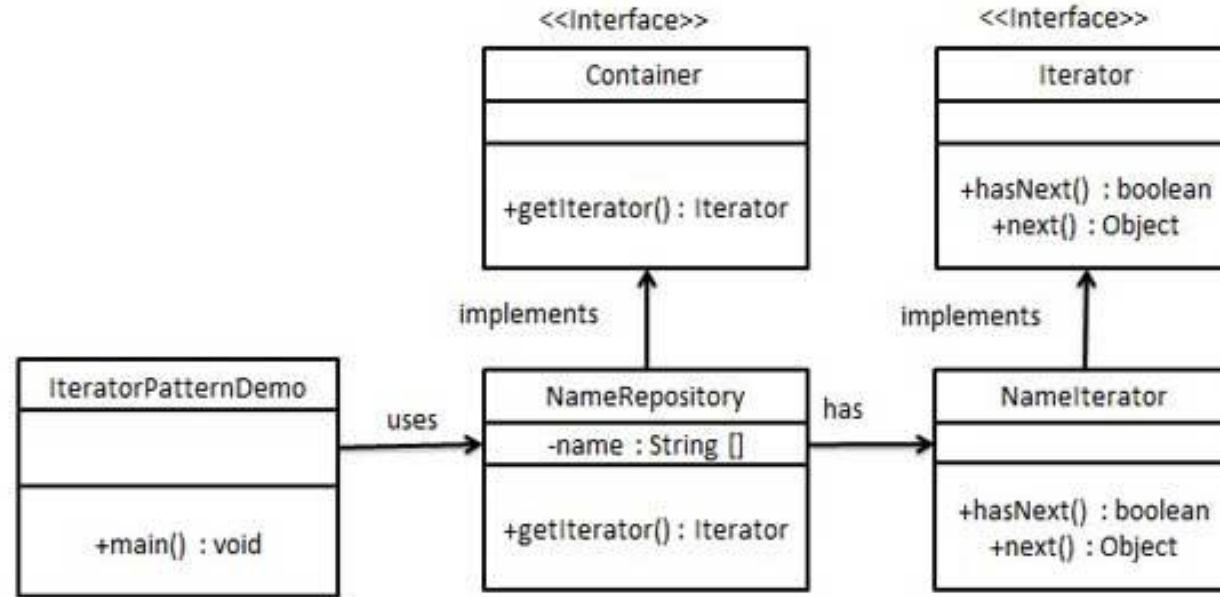
```
Stock [ Name: ABC, Quantity: 10 ] bought
Stock [ Name: ABC, Quantity: 10 ] sold
```

| 8.3 MẪU ITERATOR

8.3.1 Đặt vấn đề

- Một đối tượng tập hợp xem như là một danh sách cung cấp các phương thức truy cập các thành phần của nó. Tuy nhiên, đôi lúc chúng ta cần duyệt các thành phần của danh sách theo những cách thức và tiêu chí khác nhau nhưng không mở rộng giao diện của danh sách **List** với các phương thức cho các cách thức duyệt.
- Mẫu **Iterator** cho phép chúng ta duyệt danh sách một cách dễ dàng bằng cách tách chức năng truy cập và duyệt ra khỏi danh sách và đặt vào đối tượng **iterator**.
- Lớp **Iterator** sẽ định nghĩa một giao diện để truy cập các thành phần của danh sách, đồng thời quản lý cách thức duyệt danh sách hiện thời. Như vậy, mẫu **Iterator** cung cấp khả năng truy cập và duyệt các thành phần của một tập hợp mà không cần quan tâm đến cách thức biểu diễn bên trong.

| 8.3.2 CẤU TRÚC MẪU ITERATOR



- **Iterator** là interface gồm phương thức duyệt danh sách, kiểm tra danh sách.
- **Container** lấy duyệt từ **Iterator**, lớp cài đặt **Container** sẽ chịu trách nhiệm cài đặt **Iterator** và sử dụng nó.
- **NameIteratorDemo** sẽ sử dụng **NameRepository** để in ra danh sách.

| 8.3.3 TÌNH HUỐNG ÁP DỤNG

- Truy nhập các thành phần của một tập hợp mà không cần quan tâm đến cách thức biểu diễn bên trong.
- Hỗ trợ nhiều phương án duyệt của các đối tượng tập hợp.
- Cung cấp giao diện chung cho việc duyệt các cấu trúc tập hợp.
- **Iterator** thường được sử dụng để duyệt một cấu trúc đệ quy như **Composite**.
- Đa hình một **Iterator** dựa trên **FactoryMethod** để tạo thể nghiệm cho các lớp con tương ứng của **Iterator**.
- **Iterator** thường được sử dụng cùng với mẫu **Memento**. Một **Iterator** có thể sử dụng một **Memento** để nắm bắt trạng thái của một **Iterator** và khi đó **Iterator** lưu trữ các **memento** ở bên trong.

| 8.3.4 VÍ DỤ MẪU ITERATOR

Bước 1: Tạo interface.

Iterator.java

```
public interface Iterator {  
    public boolean hasNext();  
    public Object next();  
}
```

Container.java

```
public interface Container {  
    public Iterator getIterator();  
}
```

Bước 2: Tạo lớp cụ thể của Container.

NameRepository.java

```
public class NameRepository implements Container {  
    public String names[] = {"Robert" , "John" , "Julie" , "Lora"};  
    @Override  
    public Iterator getIterator() {  
        return new NameIterator();  
    }  
    private class NameIterator implements Iterator {  
        int index;  
        @Override  
        public boolean hasNext() {  
            if(index < names.length){  
                return true;  
            }  
            return false;  
        }  
        @Override  
        public Object next() {  
            if(this.hasNext()){  
                return names[index++];  
            }  
            return null;  
        }  
    }  
}
```

| 8.3.4 VÍ DỤ MẪU ITERATOR

Bước 3: Sử dụng NameRepository để lấy iterator và in tên.

IteratorPatternDemo.java

```
public class IteratorPatternDemo {  
  
    public static void main(String[] args) {  
        NameRepository namesRepository = new NameRepository();  
  
        for(Iterator iter = namesRepository.getIterator(); iter.hasNext());{  
            String name = (String)iter.next();  
            System.out.println("Name : " + name);  
        }  
    }  
}
```

Bước 4: Xem kết quả

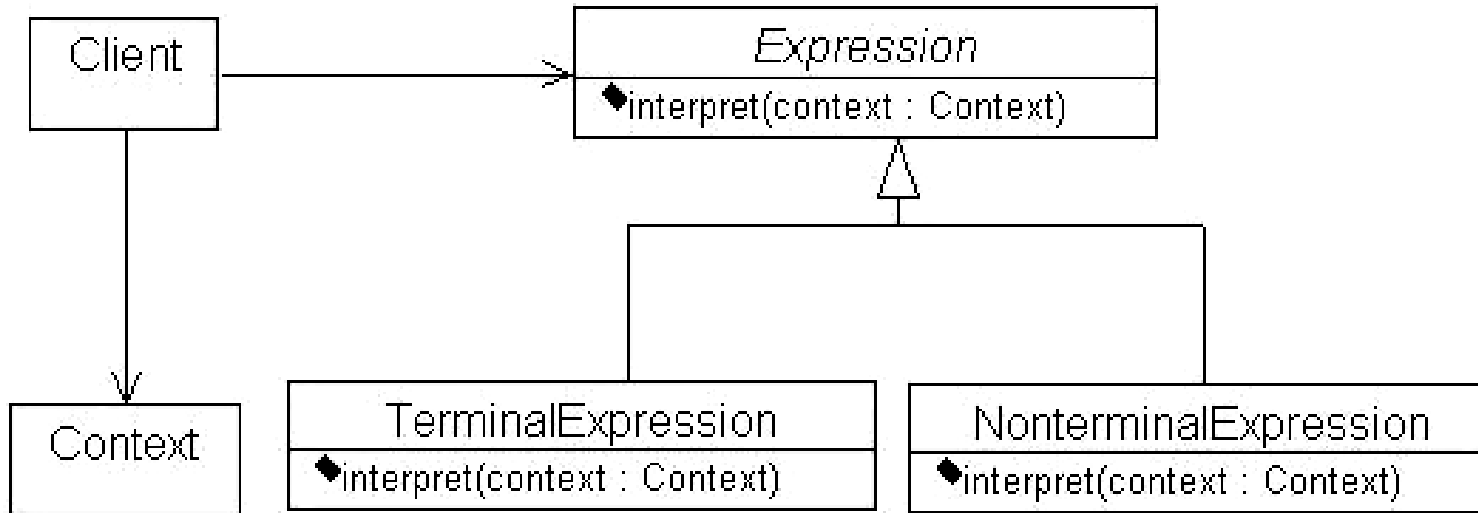
```
Name : Robert  
Name : John  
Name : Julie  
Name : Lora
```

| 8.4 MẪU INTERPRETER

8.4.1 Đặt vấn đề

- Nếu một dạng bài toán có tần suất xuất hiện tương đối lớn, người ta thường biểu diễn các thể hiện cụ thể của nó bằng các câu trong một ngôn ngữ đơn giản. Sau đó xây dựng một trình biên dịch để giải quyết bài toán bằng cách biên dịch các câu này.
- Ví dụ, tìm kiếm các xâu thoả mãn một mẫu cho trước là một bài toán thường gặp và thông thường là tạo một ngôn ngữ dùng để biểu diễn các mẫu của xâu. Thay vì xây dựng từng thuật toán riêng biệt tương ứng mỗi mẫu với một tập các xâu, người ta xây dựng một thuật toán tổng quát để có thể phiên dịch các biểu diễn thành tập các xâu tương ứng.
- Mẫu **Interpreter** mô tả cách thức xây dựng cấu trúc ngữ pháp cho những ngôn ngữ đơn giản, cách thức biểu diễn câu trong ngôn ngữ và cách thức phiên dịch các câu đó.
- **Interpreter** đưa ra một biểu diễn ngôn ngữ, xây dựng cách diễn đạt ngôn ngữ đó cùng với một trình phiên dịch sử dụng cách diễn tả trên để phiên dịch các câu.

| 8.4.2 CẤU TRÚC MẪU INTERPRETER



- **Expression**: là một giao tiếp mà thông qua nó, client tương tác với các biểu thức.
- **TerminalExpression**: là một cài đặt của giao tiếp **Expression**, đại diện cho các nốt cuối trong cây cú pháp.
- **NonterminalExpression**: là một cài đặt khác của giao tiếp **Expression**, đại diện cho các nút chưa kết thúc trong cấu trúc của cây cú pháp. Nó lưu trữ một tham chiếu đến **Expression** và triệu gọi phương thức diễn giải cho mỗi phần tử con.
- **Context**: chứa thông tin cần thiết cho một vài vị trí trong khi diễn giải. Nó có thể phục vụ như một kênh truyền thông cho các thể hiện của Expression.
- **Client**: hoặc là xây dựng hoặc là nhận một thể hiện của cây cú pháp ảo. **Client** triệu gọi các phương thức diễn giải với ngữ cảnh thích hợp khi cần thiết.

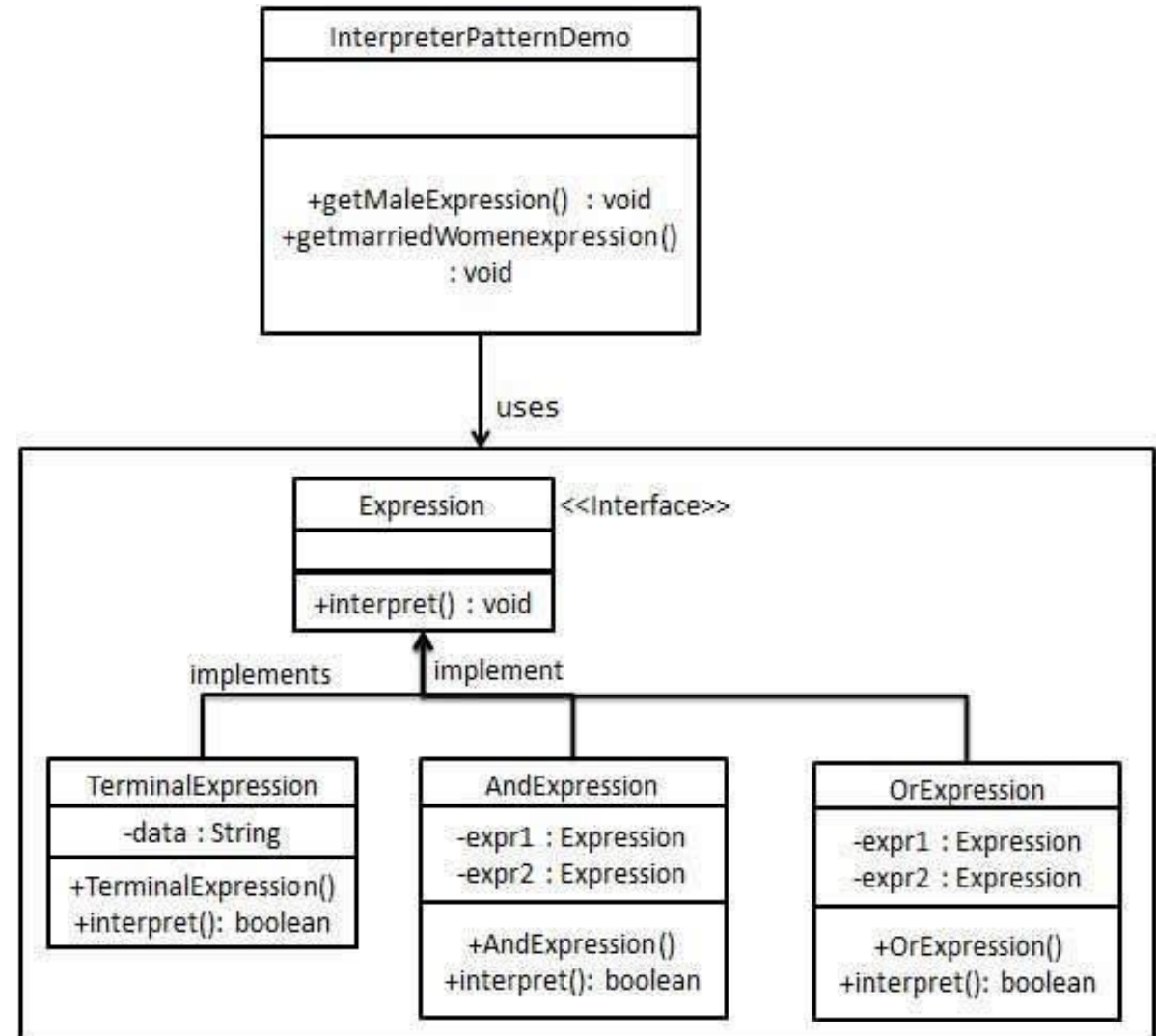
| 8.4.3 TÌNH HUỐNG ÁP DỤNG MẪU INTERPRETER

Sử dụng mẫu Interpreter khi cần phiên dịch một ngôn ngữ mà ta có thể miêu tả các câu bằng cấu trúc cây cú pháp. Mẫu này hoạt động hiệu quả nhất khi:

- Cấu trúc ngữ pháp đơn giản. Với các cấu trúc ngữ pháp phức tạp, cấu trúc lớp của ngữ pháp trở nên quá lớn và khó kiểm soát.
- Hiệu quả không phải là yếu tố quan trọng nhất. Các cách thức biên dịch hiệu quả nhất thường không áp dụng trực tiếp mẫu **Interpreter** mà phải biến đổi các biểu diễn thành các dạng khác trước.

| 8.4.4 VÍ DỤ INTERPRETER

- Chúng ta tạo interface **Expression** và các lớp cụ thể sử dụng **Expression**.
- Lớp **TerminalExpression** đóng vai trò phiên dịch chính của ngữ cảnh, lớp **OrExpression** và **AndExpression** được dùng để tạo ra các biểu thức tổ hợp.
- Lớp **InterpreterPatternDemo** sử dụng lớp **Expression** để tạo các quy tắc và demo phân tích cú pháp biểu thức.



| VÍ DỤ INTERPRETER

Bước 1: Tạo interface.

```
public interface Expression {  
    public boolean interpret(String context);  
}
```

Bước 2: Tạo các lớp cụ thể sử dụng interface.

```
public class TerminalExpression implements Expression {  
    private String data;  
    public TerminalExpression(String data){  
        this.data = data;  
    }  
    @Override  
    public boolean interpret(String context) {  
        if(context.contains(data)){  
            return true;  
        }  
        return false;  
    }  
}
```

```
public class OrExpression implements Expression {  
    private Expression expr1 = null;  
    private Expression expr2 = null;  
    public OrExpression(Expression expr1, Expression expr2) {  
        this.expr1 = expr1;  
        this.expr2 = expr2;  
    }  
    @Override  
    public boolean interpret(String context) {  
        return expr1.interpret(context) || expr2.interpret(context);  
    }  
}
```

```
public class AndExpression implements Expression {  
    private Expression expr1 = null;  
    private Expression expr2 = null;  
    public AndExpression(Expression expr1, Expression expr2) {  
        this.expr1 = expr1;  
        this.expr2 = expr2;  
    }  
    @Override  
    public boolean interpret(String context) {  
        return expr1.interpret(context) && expr2.interpret(context);  
    }  
}
```

I VÍ DỤ INTERPRETER

Bước 3: Tạo lớp InterpreterPatternDemo.

```
public class InterpreterPatternDemo {  
    //Rule: Robert and John are male  
    public static Expression getMaleExpression(){  
        Expression robert = new TerminalExpression("Robert");  
        Expression john = new TerminalExpression("John");  
        return new OrExpression(robert, john);  
    }  
    //Rule: Julie is a married women  
    public static Expression getMarriedWomanExpression(){  
        Expression julie = new TerminalExpression("Julie");  
        Expression married = new TerminalExpression("Married");  
        return new AndExpression(julie, married);  
    }  
    public static void main(String[] args) {  
        Expression isMale = getMaleExpression();  
        Expression isMarriedWoman = getMarriedWomanExpression();  
        System.out.println("John is male? " + isMale.interpret("John"));  
        System.out.println("Julie is a married women? " + isMarriedWoman.interpret("Married Julie"));  
    }  
}
```

Bước 4: Xem kết quả.

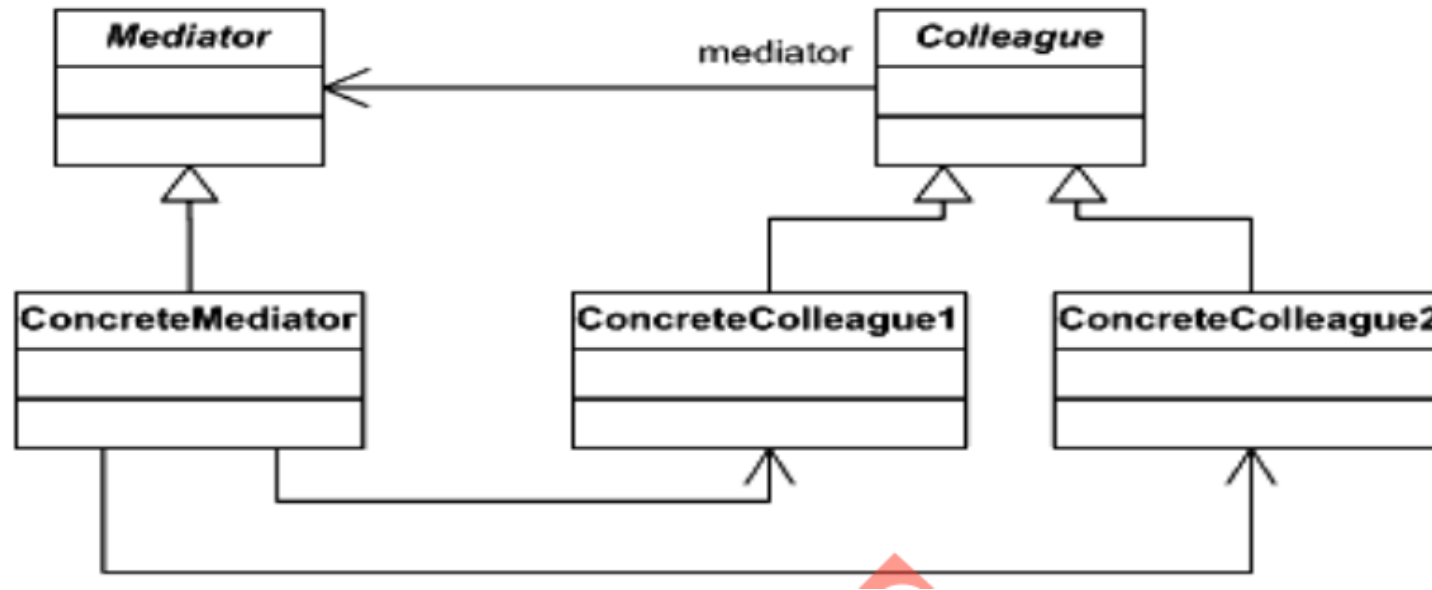
```
John is male? true  
Julie is a married women? true
```

| 8.5 MẪU MEDIATOR

8.5.1 Đặt vấn đề

- Cách thiết kế hướng đối tượng khuyến khích việc phân bố các hành vi giữa các đối tượng. Tuy nhiên, việc phân chia như vậy có thể dẫn đến một cấu trúc có nhiều liên kết giữa các đối tượng và trong trường hợp xấu nhất là tất cả các đối tượng đều có liên kết trực tiếp với nhau.
- Mẫu **Mediator** được dùng để **đóng gói cách thức tương tác của một tập hợp các đối tượng** và như vậy giảm bớt liên kết và cho phép thay đổi cách thức tương tác giữa các đối tượng một cách linh hoạt.
- Mẫu **Mediator** có ưu điểm là giảm việc chia lớp con và liên kết giữa các đối tượng. Nó cũng làm đơn giản hoá giao tiếp giữa đối tượng bằng cách thay các tương tác nhiều - nhiều bằng tương tác 1- nhiều giữa nó và các đối tượng khác.
- Đồng thời nó trừu tượng hoá cách thức cộng tác giữa các đối tượng và tạo ra sự tách biệt rõ ràng hơn giữa các tương tác ngoài và các đặc tính bên trong của đối tượng.

| 8.5.2 CẤU TRÚC MẪU MEDIATOR

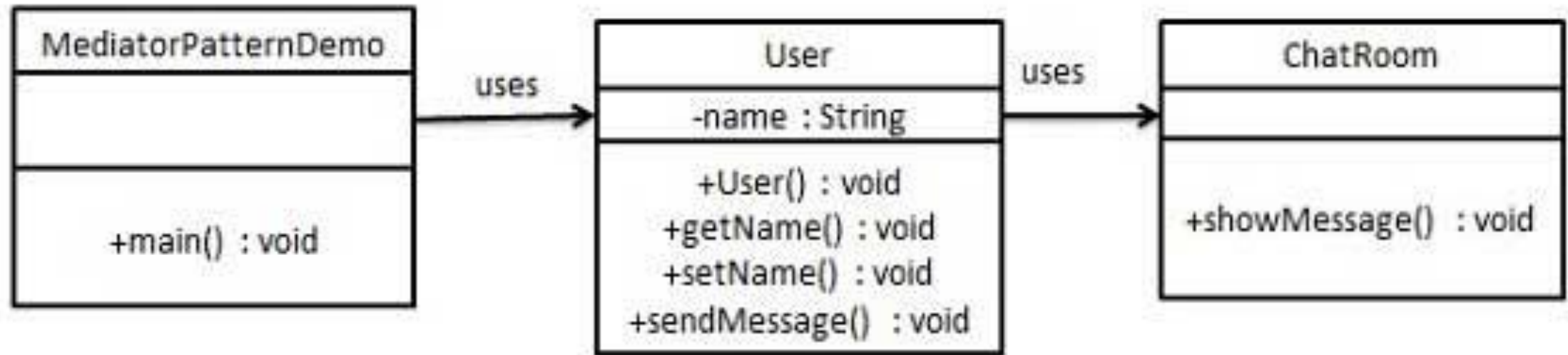


- **Mediator** (IChatroom): định nghĩa một giao tiếp cho các đối tượng cộng tác.
- **ConcreteMediator** (Chatroom) Xây dựng các hành vi tương tác giữa các đối tượng colleague và xác định các đối tượng colleague.
- **Colleague classes** (Participant) mỗi lớp **Colleague** đều xác định đối tượng **Mediator** tương ứng. Mỗi đối tượng colleague trao đổi với đối tượng mediator khi muốn trao đổi với colleague khác.

| 8.5.3 TÌNH HUỐNG ÁP DỤNG

- Một nhóm các đối tượng trao đổi thông tin một cách rõ ràng nhưng khá phức tạp và điều này có thể dẫn đến hệ thống có các kết nối phi cấu trúc và khó hiểu.
- Việc sử dụng lại một đối tượng gặp khó khăn vì nó liên kết với quá nhiều đối tượng khác. Mẫu này cho phép tùy biến một ứng xử được phân tán trong vài lớp mà không phải phân nhiều lớp con.
- **Mediator** khác với **Facade** ở chỗ **Facade** trừu tượng một hệ thống con của các đối tượng để cung cấp một giao diện tiện ích hơn. Ngược lại, **Mediator** cho phép kết hợp các hành vi mà các đối tượng cộng tác không thể cung cấp.

| 8.5.4 VÍ DỤ MEDIATOR



- Một chatroom có nhiều users có thể gửi tin nhắn đến chatroom và chatroom hiển thị tin nhắn cho tất cả users.
- Chúng ta tạo 2 lớp **ChatRoom**, **User**. **User** sử dụng phương thức của **ChatRoom** để chia sẻ tin nhắn.
- Lớp **MediatorPatternDemo** sử dụng **User** để hiển thị demo ứng dụng.

| 8.5.4 VÍ DỤ MEDIATOR

Bước 1: Tạo lớp mediator.

```
import java.util.Date;

public class ChatRoom {
    public static void showMessage(User user, String message){
        System.out.println(new Date().toString() + " [" + user.getName() + "] : " + message);
    }
}
```

| VÍ DỤ MEDIATOR

Bước 2: Tạo lớp User .

```
public class User {  
    private String name;  
    public String getName() {  
        return name;  
    }  
    public void setName(String name) {  
        this.name = name;  
    }  
    public User(String name){  
        this.name = name;  
    }  
    public void sendMessage(String message){  
        ChatRoom.showMessage(this,message);  
    }  
}
```

Bước 3: Tạo lớp MediatorPatternDemo .

```
public class MediatorPatternDemo {  
    public static void main(String[] args) {  
        User robert = new User("Robert");  
        User john = new User("John");  
  
        robert.sendMessage("Hi! John!");  
        john.sendMessage("Hello! Robert!");  
    }  
}
```

Bước 4: Xem kết quả.

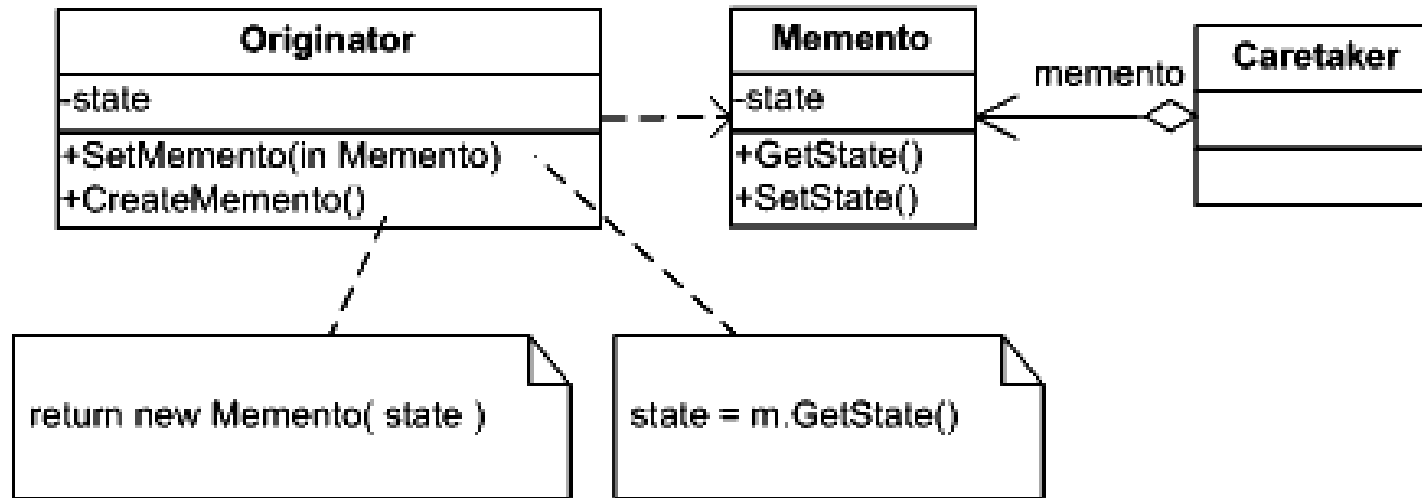
```
Thu Jan 31 16:05:46 IST 2013 [Robert] : Hi! John!  
Thu Jan 31 16:05:46 IST 2013 [John] : Hello! Robert!
```

| 8.6 MẪU MEMENTO

8.6.1 Đặt vấn đề

- Đôi lúc, việc lưu lại trạng thái của một đối tượng là cần thiết. Ví dụ khi xây dựng cơ chế checkpoints và undo để phục hồi hệ thống khỏi trạng thái lỗi. Tuy nhiên, các đối tượng thường phải che dấu một vài hoặc tất cả các thông tin trạng thái của mình, làm cho chúng không thể được truy cập từ bên ngoài. Vấn đề này có thể giải quyết với mẫu **Memento**.
- **Memento** là đối tượng có chức năng lưu lại trạng thái nội tại của một đối tượng khác. Cơ chế undo sẽ yêu cầu một **memento** ban đầu khi nó cần khôi phục lại trạng thái ban đầu của đối tượng.
- Chỉ có đối tượng ban đầu mới có quyền truy xuất và lưu trữ thông tin vào memento do nó trong suốt đối với các đối tượng còn lại. Như vậy, **Memento** là mẫu thiết kế có thể lưu lại trạng thái của một đối tượng để khôi phục lại sau này mà không vi phạm nguyên tắc đóng gói.

| 8.6.2 CẤU TRÚC MẪU MEMENTO



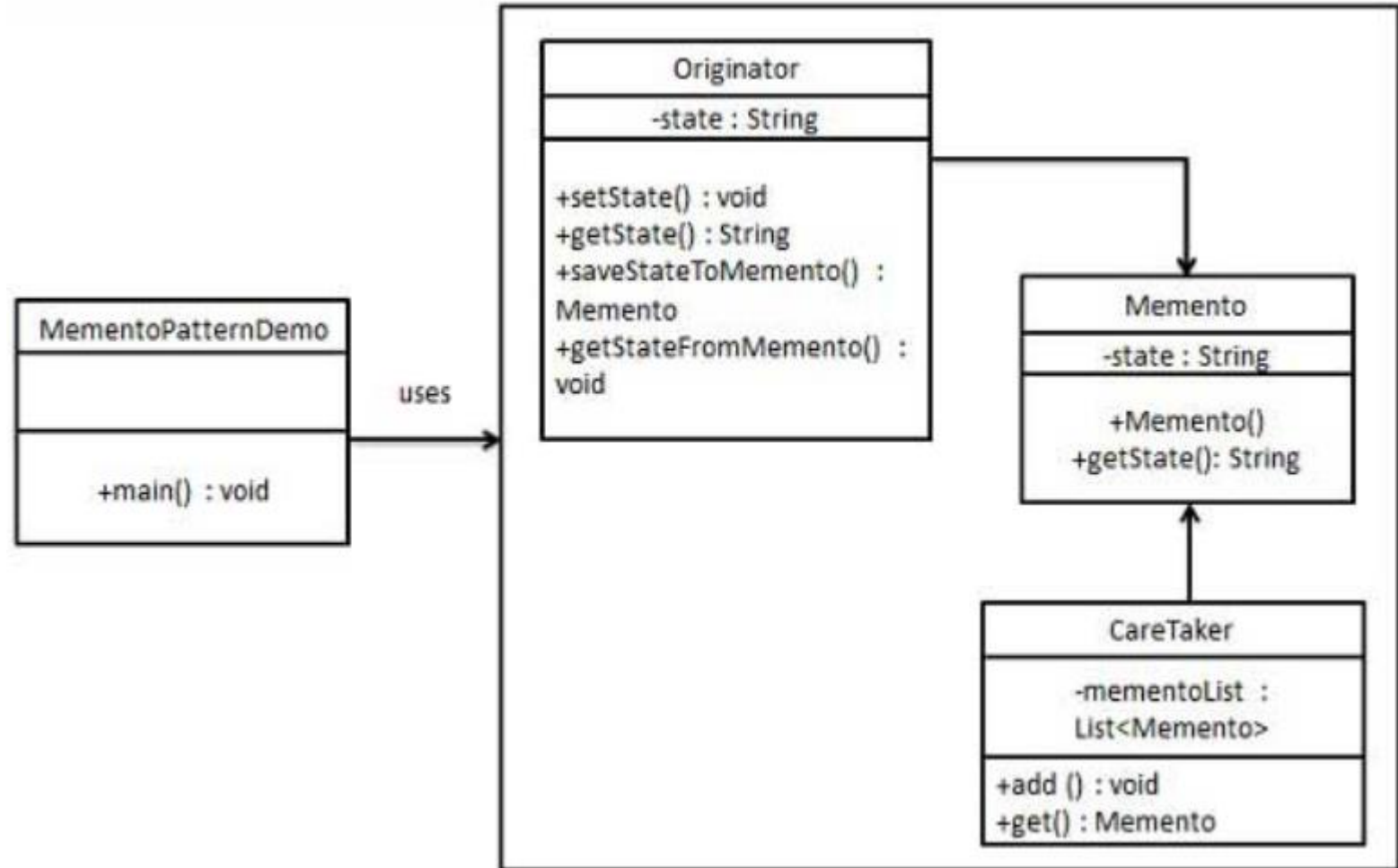
- **Memento**: lưu trữ trạng thái của đối tượng **Originator** và bảo vệ, chống truy cập từ các đối tượng khác **Originator**.
- **Originator**: Tạo **memento** chứa bản chụp trạng thái của mình và sử dụng **memento** để khôi phục về trạng thái cũ.
- **Caretaker**: Có trách nhiệm quản lý các **memento**, nó không thay đổi hoặc truy xuất nội dung của **memento**.

| 8.6.3 TÌNH HUỐNG ÁP DỤNG MEMENTO

- Cần lưu lại trạng thái nội bộ của một đối tượng để có thể khôi phục trạng thái đó sau này. Việc xây dựng giao diện trực tiếp để truy xuất thông tin trạng thái sẽ làm lộ diện cấu trúc và phá hỏng tính đóng gói của đối tượng.
- Các **Command** có thể sử dụng các memento để duy trì trạng thái cho các thao tác có khả năng phục hồi được.
- Chi phí ẩn của việc lưu trữ **memento**: caretaker có nhiệm vụ quản lý cũng như xóa bỏ các memento mà nó yêu cầu tạo ra. Tuy nhiên nó không biết được kích thước của memento là bao nhiêu và do đó có thể tốn nhiều không gian bộ nhớ khi lưu trữ **memento**.

| 8.6.4 VÍ DỤ MEMENTO

- **Memento** chứa trạng thái của đối tượng cần được khôi phục.
- Originator tạo và lưu trữ trạng thái trong đối tượng **Memento** và **Caretaker** chịu trách nhiệm khôi phục trạng thái từ **Memento**.
- Chúng ta có các lớp **Memento**, **Originator**, **Caretaker**.



| 8.6.4 VÍ DỤ MEMENTO

Bước 1: Tạo lớp Memento.

```
public class Memento {  
    private String state;  
  
    public Memento(String state){  
        this.state = state;  
    }  
  
    public String getState(){  
        return state;  
    }  
}
```

Bước 2: Tạo lớp Originator.

```
public class Originator {  
    private String state;  
  
    public void setState(String state){  
        this.state = state;  
    }  
  
    public String getState(){  
        return state;  
    }  
  
    public Memento saveStateToMemento(){  
        return new Memento(state);  
    }  
  
    public void getStateFromMemento(Memento memento){  
        state = memento.getState();  
    }  
}
```

| 8.6.4 VÍ DỤ MEMENTO

Bước 3: Tạo lớp CareTaker .

```
import java.util.ArrayList;
import java.util.List;

public class CareTaker {
    private List<Memento> mementoList = new ArrayList<Memento>();

    public void add(Memento state){
        mementoList.add(state);
    }

    public Memento get(int index){
        return mementoList.get(index);
    }
}
```

| 8.6.4 VÍ DỤ MEMENTO

Bước 4: Sử dụng CareTaker và Originator.

```
public class MementoPatternDemo {  
    public static void main(String[] args) {  
  
        Originator originator = new Originator();  
        CareTaker careTaker = new CareTaker();  
  
        originator.setState("State #1");  
        originator.setState("State #2");  
        careTaker.add(originator.saveStateToMemento());  
  
        originator.setState("State #3");  
        careTaker.add(originator.saveStateToMemento());  
  
        originator.setState("State #4");  
        System.out.println("Current State: " + originator.getState());  
  
        originator.getStateFromMemento(careTaker.get(0));  
        System.out.println("First saved State: " + originator.getState());  
        originator.getStateFromMemento(careTaker.get(1));  
        System.out.println("Second saved State: " + originator.getState());  
    }  
}
```

Bước 5: Xem kết quả

Current State: State #4
First saved State: State #2
Second saved State: State #3

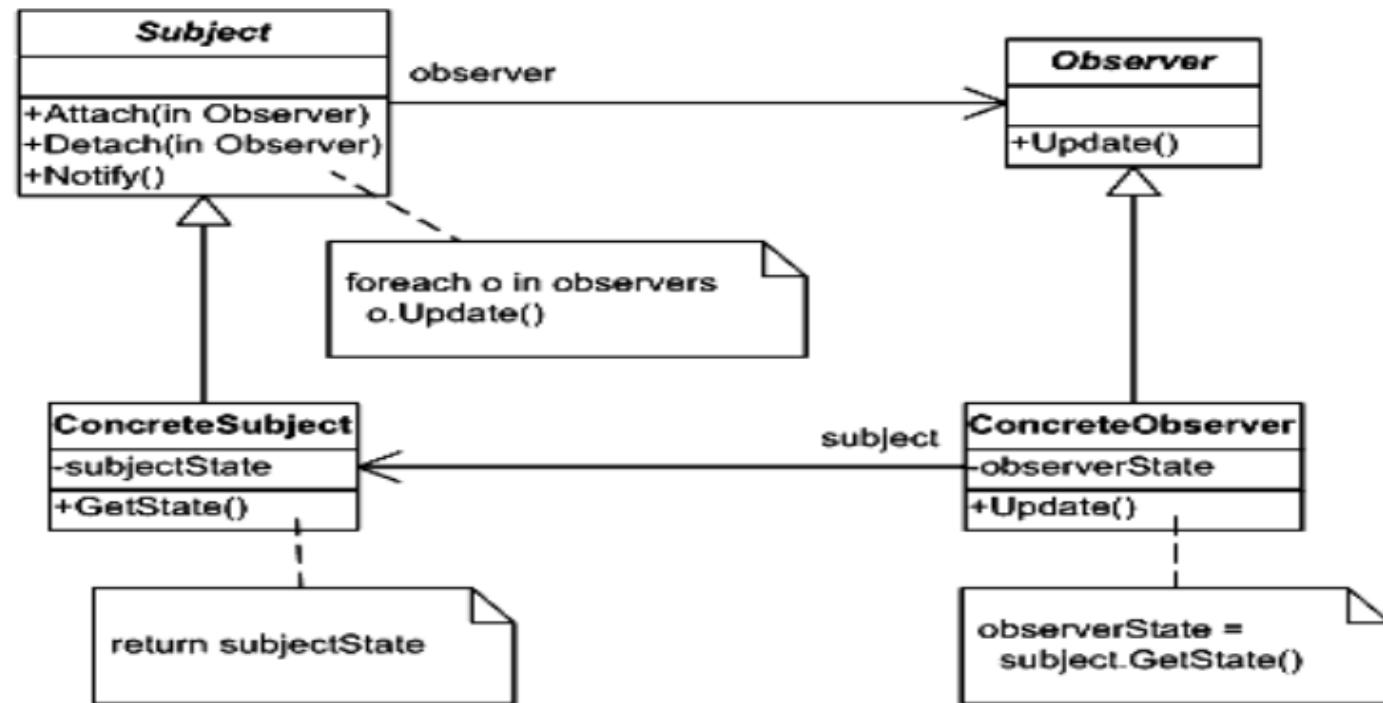
| 8.7 MẪU OBSERVER

8.7.1 Đặt vấn đề

- Mẫu **Observer** ích lợi trong thiết kế mô hình giao tiếp giữa một đối tượng và một tập đối tượng phụ thuộc nhau. Nó cho phép đồng bộ hóa trạng thái của tập đối tượng với đối tượng kia và định nghĩa phụ thuộc 1 - nhiều giữa các đối tượng để khi một đối tượng thay đổi trạng thái thì tất cả các phụ thuộc của nó được nhận biết và cập nhật tự động.
- **Observer** pattern là một mẫu thiết kế phần mềm mà một đối tượng, gọi là **subject**, duy trì một danh sách các thành phần phụ thuộc nó, gọi là **observer**, và thông báo tới chúng một cách tự động về bất cứ thay đổi nào.

| 8.7.2 CẤU TRÚC MẪU OBSERVER

- **Observer**: Định nghĩa một giao diện cập nhật cho các đối tượng mà sẽ nhận tín hiệu của sự thay đổi tại chủ thể.
- **Subject**: hiểu về các Observer của nó. Một số lượng bất kỳ Observer có thể theo dõi một chủ thể nào đó và cung cấp một giao diện cho việc gắn và tách các đối tượng Observer.
- **ConcreteSubject**: Lưu trữ trạng thái của **ConcreteObserver** cần quan tâm và gửi tín hiệu đến các observer của nó khi trạng thái của nó thay đổi.
- **ConcreteObserver**: Duy trì một tham chiếu tới một đối tượng **ConcreteSubject** và lưu trữ các trạng thái cố định. Nó cài đặt giao diện cập nhật của **Observer** để giữ các trạng thái cố định của nó.

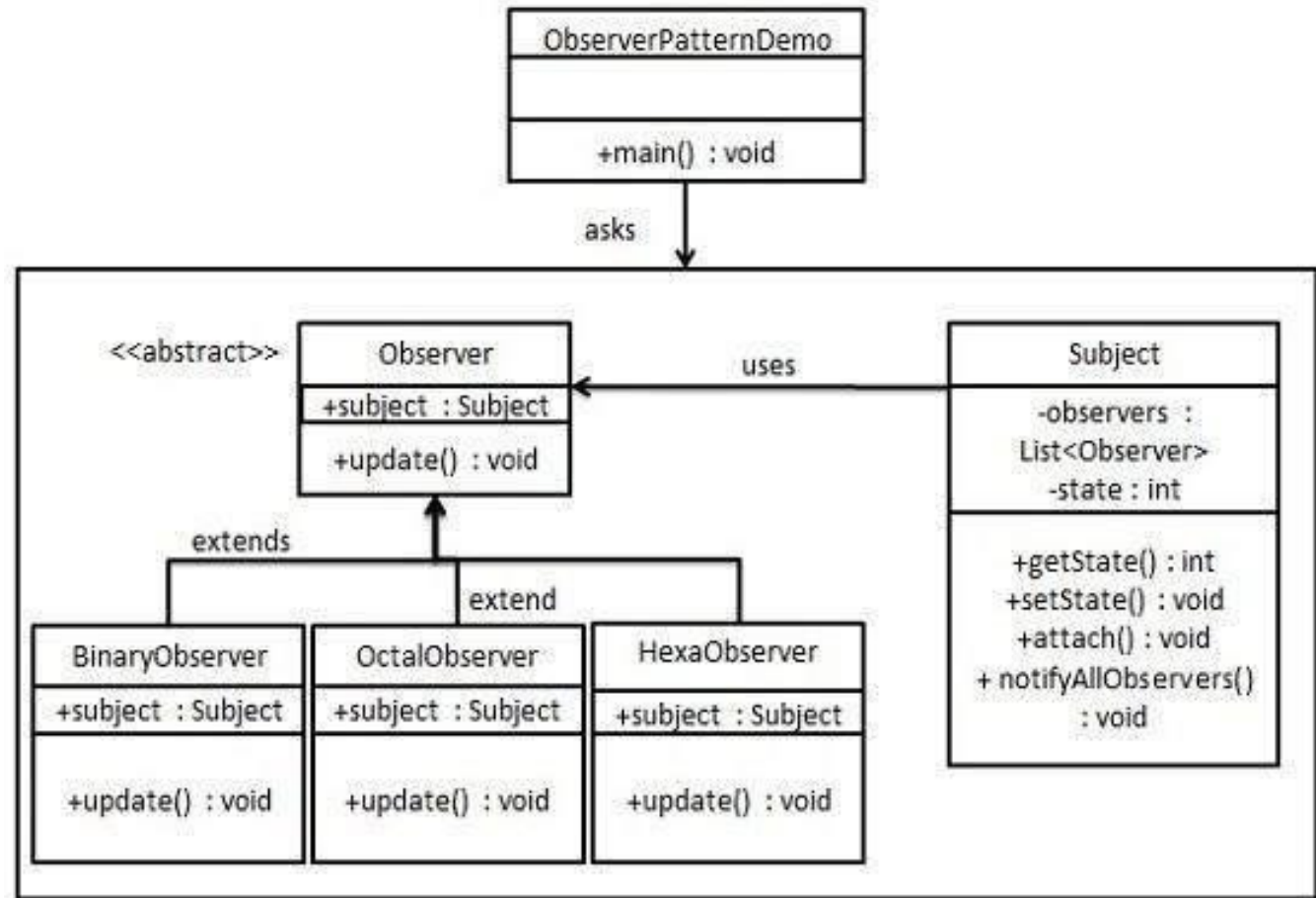


| 8.7.3 TÌNH HUỐNG ÁP DỤNG OBSERVER

- **Observer Pattern** thường được sử dụng trong các ứng dụng thời gian thực như Live Systems, Stock Market, và các trò chơi trực tuyến. Nó cũng rất hữu ích trong việc xây dựng các ứng dụng có tính năng "Push Notification", là nơi một số sự kiện nhất định sẽ thông báo tới người dùng.

| 8.7.4 VÍ DỤ MẪU OBSERVER

- Lớp trừu tượng **Observer** có lớp mở rộng là **BinaryObserver**, **OctalObserver**, **HexaObserver**.
- **Subject** là một đối tượng có các phương thức để thêm và xóa observer khỏi client.
- Lớp **ObserverPatternDemo** sử dụng **Subject** và đối tượng cụ thể để demo ứng dụng.



| 8.7.4 VÍ DỤ MẪU OBSERVER

Bước 1: Tạo lớp Observer.

```
public abstract class Observer {  
    protected Subject subject;  
    public abstract void update();  
}
```

Bước 2: Tạo lớp Subject.

```
import java.util.ArrayList;  
import java.util.List;  
public class Subject {  
    private List<Observer> observers = new ArrayList<Observer>();  
    private int state;  
    public int getState() {  
        return state;  
    }  
    public void setState(int state) {  
        this.state = state;  
        notifyAllObservers();  
    }  
    public void attach(Observer observer){  
        observers.add(observer);  
    }  
    public void notifyAllObservers(){  
        for (Observer observer : observers) {  
            observer.update();  
        }  
    }  
}
```

| 8.7.4 VÍ DỤ MẪU OBSERVER

Bước 3: Tạo các lớp mở rộng của Observer.

```
public class BinaryObserver extends Observer{
    public BinaryObserver(Subject subject){
        this.subject = subject;
        this.subject.attach(this);
    }
    @Override
    public void update() {
        System.out.println("Binary String: " +Integer.toBinaryString(subject.getState()));
    }
}
```

```
public class OctalObserver extends Observer{
    public OctalObserver(Subject subject){
        this.subject = subject;
        this.subject.attach(this);
    }
    @Override
    public void update() {
        System.out.println("Octal String: " + Integer.toOctalString( subject.getState()));
    }
}
```

| 8.7.4 VÍ DỤ MẪU OBSERVER

Bước 3: Tạo các lớp mở rộng của Observer.

```
public class HexaObserver extends Observer{

    public HexaObserver(Subject subject){
        this.subject = subject;
        this.subject.attach(this);
    }

    @Override
    public void update() {
        System.out.println("Hex String:"+Integer.toHexString(subject.getState()).toUpperCase());}
}
```

| 8.7.4 VÍ DỤ MẪU OBSERVER

Bước 4: Sử dụng Subject và các đối tượng observer.

```
public class ObserverPatternDemo {  
    public static void main(String[] args) {  
        Subject subject = new Subject();  
  
        new HexaObserver(subject);  
        new OctalObserver(subject);  
        new BinaryObserver(subject);  
  
        System.out.println("First state change: 15");  
        subject.setState(15);  
        System.out.println("Second state change: 10");  
        subject.setState(10);  
    }  
}
```

Bước 5: Xem kết quả

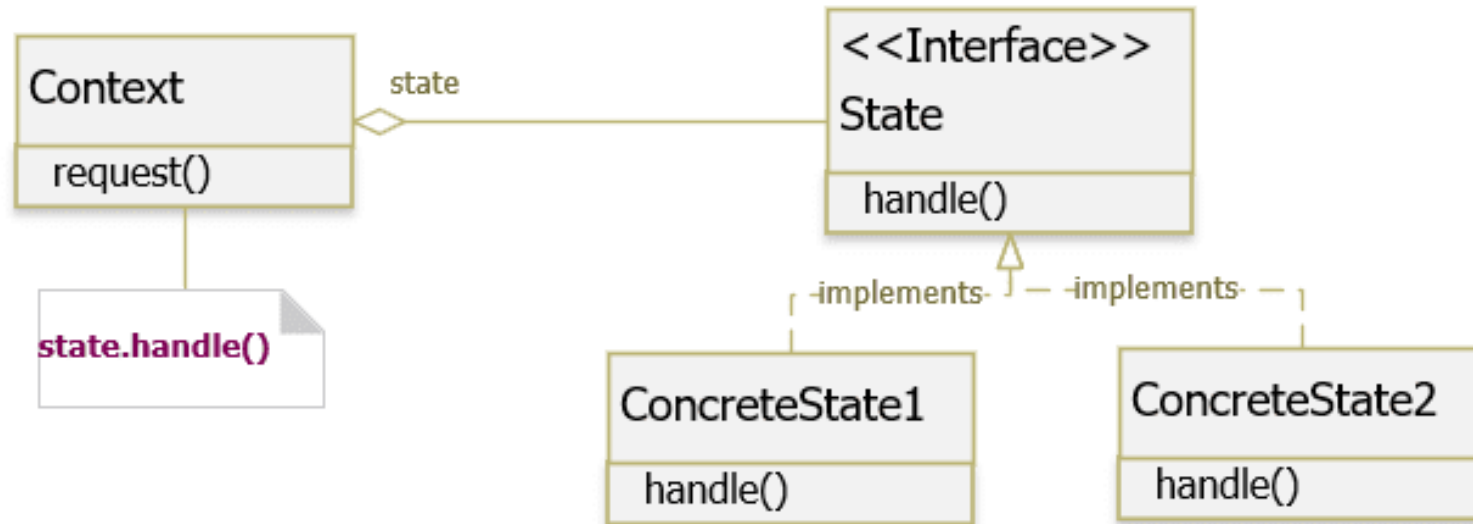
```
First state change: 15  
Hex String: F  
Octal String: 17  
Binary String: 1111  
Second state change: 10  
Hex String: A  
Octal String: 12  
Binary String: 1010
```

| 8.8 MẪU STATE

8.8.1 Đặt vấn đề

- Trong thực tế, có rất nhiều thứ tại bất kỳ thời điểm nào cũng có một số trạng thái hữu hạn khác nhau, và chúng có thể chuyển từ trạng thái này sang trạng thái khác. Trong các trạng thái khác nhau đó, chúng có những đặc tính và hành vi khác nhau
- Mẫu state có thể sử dụng trong thiết kế cấu trúc của lớp để cho các thể hiện của lớp có thể tồn tại ở các trạng thái khác nhau và thể hiện hành vi khác nhau tùy theo trạng thái.
- State Pattern là một mẫu thiết kế hành vi cho phép một object thay đổi hành vi của nó khi trạng thái bên trong của nó thay đổi.

| 8.8.2 CẤU TRÚC MẪU STATE

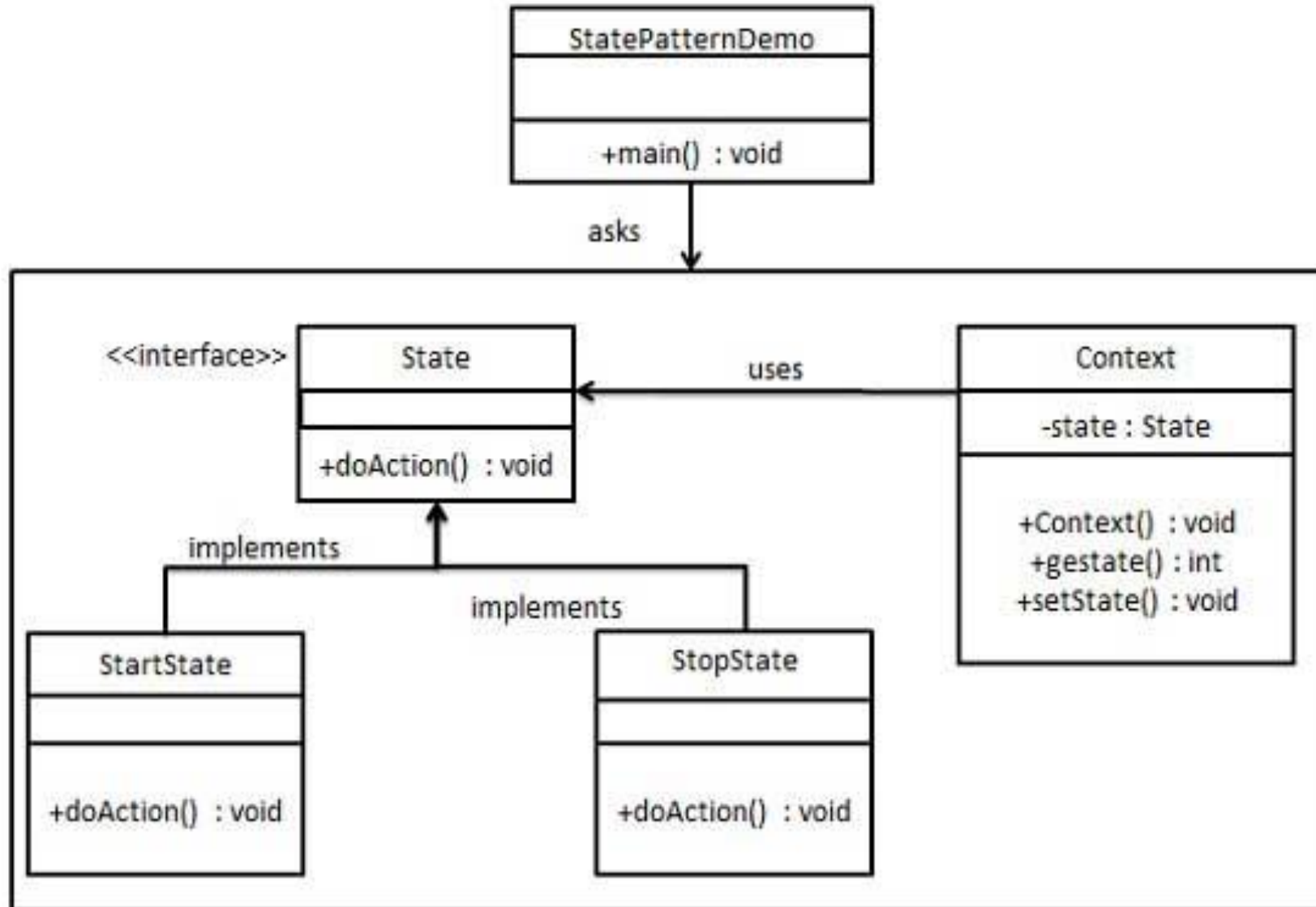


- **Context** : được sử dụng bởi Client. Client không truy cập trực tiếp đến **State** của đối tượng. Lớp **Context** này chứa thông tin của **ConcreteState** object, cho hành vi nào tương ứng với trạng thái nào hiện đang được thực hiện.
- **State** : là một interface hoặc abstract class xác định các đặc tính cơ bản của tất cả các đối tượng **ConcreteState**. Chúng sẽ được sử dụng bởi đối tượng **Context** để truy cập chức năng có thể thay đổi.
- **ConcreteState**: cài đặt các phương thức của **State**. Mỗi **ConcreteState** có thể thực hiện logic và hành vi của riêng nó tùy thuộc vào **Context**.

| 8.8.3 TÌNH HUỐNG ÁP DỤNG STATE

- Sử dụng State khi có một object hoạt động khác nhau tùy thuộc vào trạng thái hiện tại của nó, số lượng trạng thái là rất lớn và mã của trạng thái cụ thể thường xuyên thay đổi.
- Sử dụng State khi có một lớp với nhiều các điều kiện lớn làm thay đổi cách class hoạt động theo các giá trị hiện tại của các trường của class.
- Sử dụng State khi có nhiều mã trùng lặp qua các trạng thái và chuyển đổi tương tự của State Pattern dựa trên điều kiện.

| 8.8.4 VÍ DỤ STATE



| 8.8.4 VÍ DỤ STATE

Bước 1: Tạo interface.

```
public interface State {  
    public void doAction(Context context);  
}
```

Bước 2: Tạo lớp cụ thể sử dụng interface.

```
public class StartState implements State {  
    public void doAction(Context context) {  
        System.out.println("Player is in start state");  
        context.setState(this);  
    }  
    public String toString(){  
        return "Start State";  
    }  
}
```

```
public class StopState implements State {  
    public void doAction(Context context) {  
        System.out.println("Player is in stop state");  
        context.setState(this);  
    }  
    public String toString(){  
        return "Stop State";  
    }  
}
```

| 8.8.4 VÍ DỤ STATE

Bước 3: Tạo lớp Context.

```
public class Context {  
    private State state;  
    public Context(){  
        state = null;  
    }  
    public void setState(State state){  
        this.state = state;  
    }  
    public State getState(){  
        return state;  
    }  
}
```

Bước 5: Xem kết quả.

Player is in start state
Start State
Player is in stop state
Stop State

Bước 4: Sử dụng Context để xem hành vi khi State thay đổi.

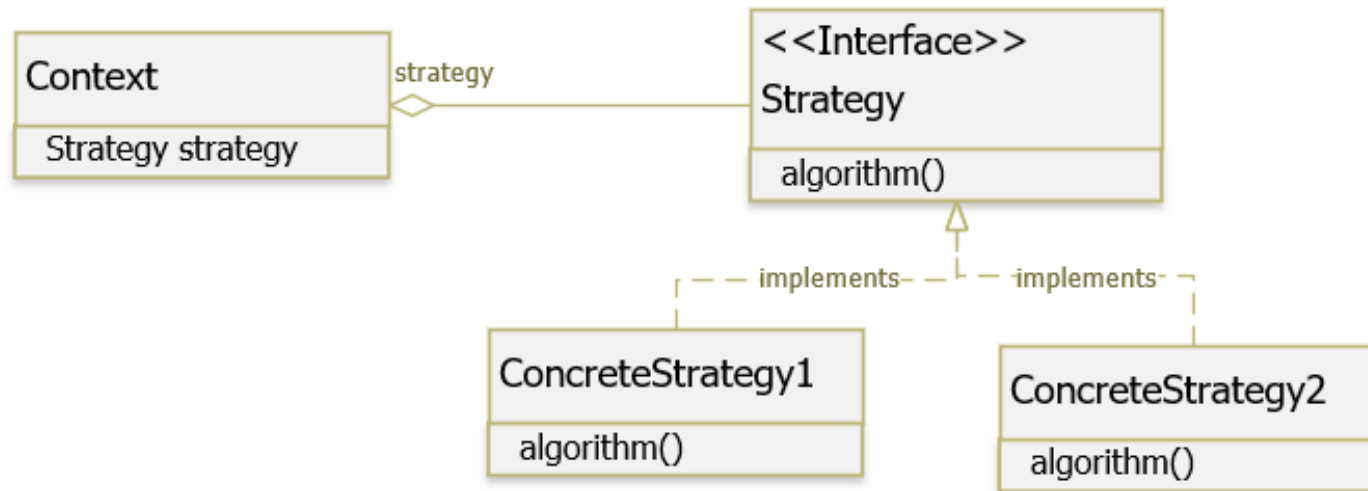
```
public class StatePatternDemo {  
    public static void main(String[] args) {  
        Context context = new Context();  
  
        StartState startState = new StartState();  
        startState.doAction(context);  
  
        System.out.println(context.getState().toString());  
  
        StopState stopState = new StopState();  
        stopState.doAction(context);  
  
        System.out.println(context.getState().toString());  
    }  
}
```

| 8.9 MẪU STRATEGY

8.9.1 Đặt vấn đề

- **Strategy** là mẫu thiết kế được sử dụng khi một đối tượng client cần chọn một thuật toán thích hợp từ một tập thuật toán có liên quan với nhau. Mẫu này đề nghị cài đặt mỗi thuật toán trong một lớp tách biệt. Mỗi thuật toán đóng gói trong một lớp tách biệt gọi là một chiến lược **strategy** và đối tượng sử dụng đối tượng chiến lược gọi là đối tượng ngữ cảnh **context** object. Như vậy, với những đối tượng chiến lược khác nhau, việc thay đổi hành vi của đối tượng ngữ cảnh đơn giản là thay đổi đối tượng chiến lược của nó thành đối tượng cài đặt thuật toán mong muốn.
- Để giúp cho đối tượng ngữ cảnh truy nhập các đối tượng chiến lược khác nhau, thì tất cả đối tượng chiến lược phải được thiết kế với cùng giao tiếp.
- **Strategy Pattern** được phát biểu như sau: "Xác định một họ chức năng, gói gọn từng chức năng và làm cho chúng có thể thay thế cho nhau". Khi áp dụng Strategy Pattern thì các hành vi hoặc giải thuật của một class có thể thay đổi ở thời điểm chạy.

| 8.9.2 CẤU TRÚC MẪU STRATEGY

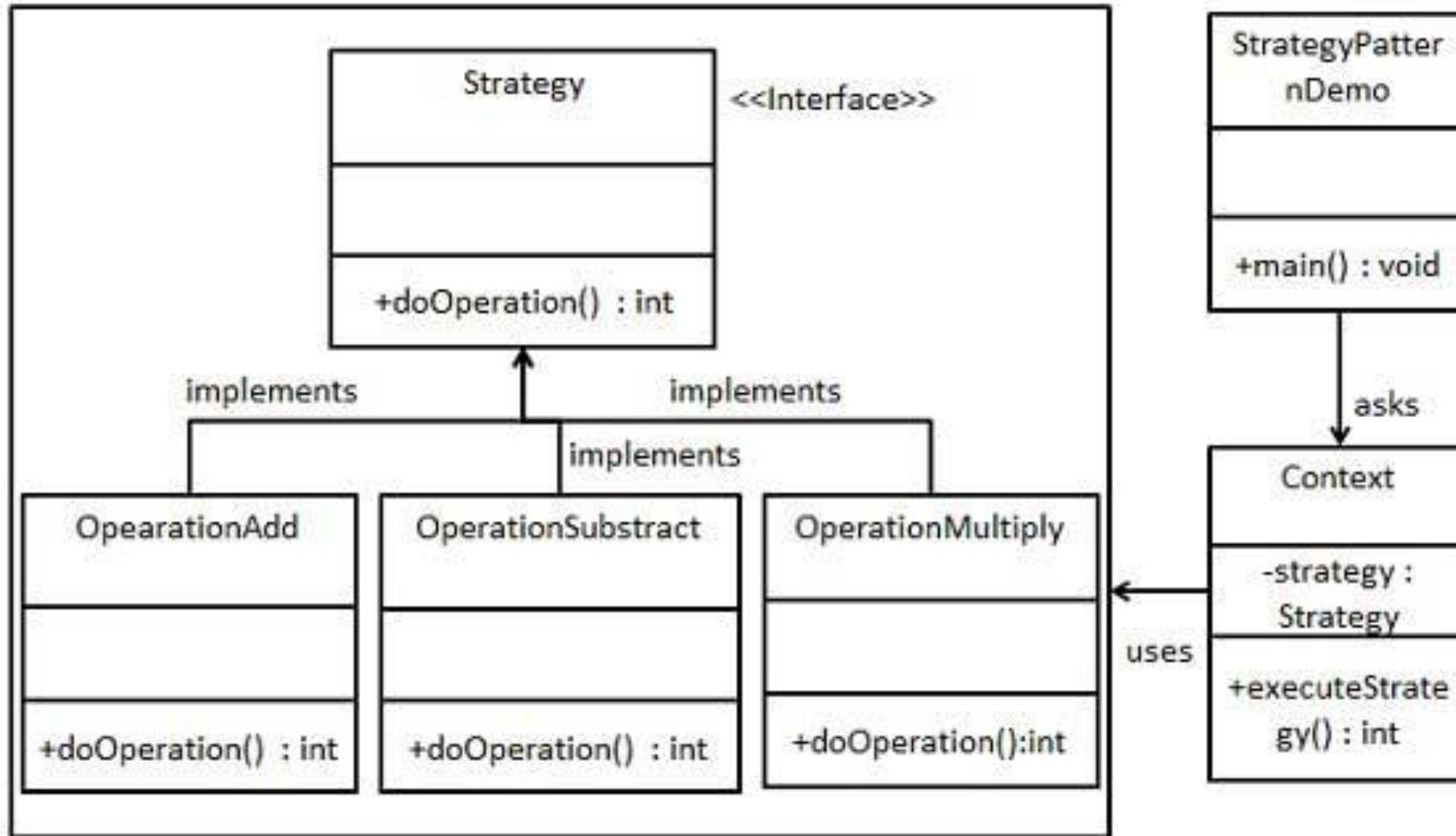


- **Strategy** : định nghĩa các hành vi có thể có của một **Strategy**.
- **ConcreteStrategy**: cài đặt các hành vi cụ thể của **Strategy**.
- **Context** : chứa một tham chiếu đến đối tượng **Strategy** và nhận các yêu cầu từ **Client**, các yêu cầu này sau đó được ủy quyền cho **Strategy** thực hiện.

| 8.9.3 TÌNH HUỐNG ÁP DỤNG STRATEGY

- Có một đoạn code dễ thay đổi, và tách chúng ra khỏi chương trình chính để dễ dàng bảo trì.
- Muốn tránh sự rắc rối, khi phải hiện thực một chức năng nào đó qua quá nhiều lớp con.
- Muốn thay đổi thuật toán sử dụng khi chạy chương trình.

| 8.9.4 VÍ DỤ STRATEGY



| 8.9.4 VÍ DỤ STRATEGY

Bước 1: Tạo interface.

```
public interface Strategy {  
    public int doOperation(int num1, int num2);  
}
```

Bước 2: Tạo các lớp cụ thể.

```
public class OperationAdd implements Strategy{  
    @Override  
    public int doOperation(int num1, int num2) {  
        return num1 + num2;  
    }  
}
```

```
public class OperationSubtract implements Strategy{  
    @Override  
    public int doOperation(int num1, int num2) {  
        return num1 - num2;  
    }  
}
```

```
public class OperationMultiply implements Strategy{  
    @Override  
    public int doOperation(int num1, int num2) {  
        return num1 * num2;  
    }  
}
```

| 8.9.4 VÍ DỤ STRATEGY

Bước 3: Tạo lớp Context.

```
public class Context {  
    private Strategy strategy;  
    public Context(Strategy strategy){  
        this.strategy = strategy;  
    }  
    public int executeStrategy(int num1, int num2){  
        return strategy.doOperation(num1, num2);  
    }  
}
```

Bước 5: Xem kết quả.

```
10 + 5 = 15  
10 - 5 = 5  
10 * 5 = 50
```

Bước 4: Tạo lớp demo.

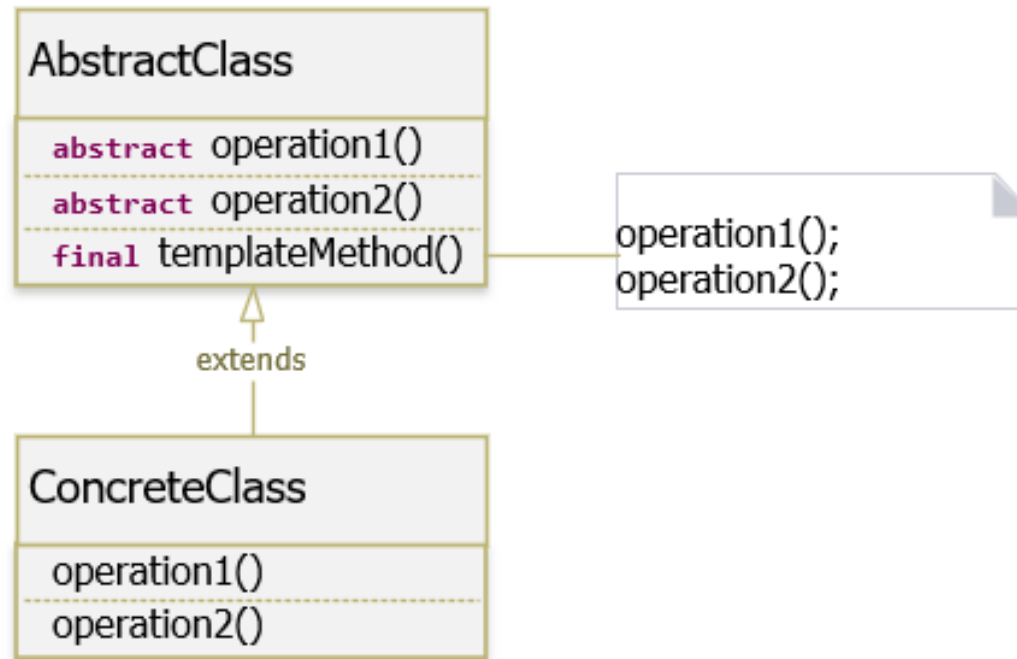
```
public class StrategyPatternDemo {  
    public static void main(String[] args) {  
        Context context = new Context(new OperationAdd());  
        System.out.println("10 + 5 = " + context.executeStrategy(10, 5));  
  
        context = new Context(new OperationSubtract());  
        System.out.println("10 - 5 = " + context.executeStrategy(10, 5));  
  
        context = new Context(new OperationMultiply());  
        System.out.println("10 * 5 = " + context.executeStrategy(10, 5));  
    }  
}
```

| 8.10 MẪU TEMPLATE METHOD

8.10.1 Đặt vấn đề

- Phương thức khung (**Template Method**) có thể được sử dụng trong những tình huống khi có một thuật toán mà một số bước của nó có thể cài đặt theo nhiều cách khác nhau.
- Mẫu này cho rằng khi đó nên đưa sườn thuật toán thành một phương thức tách biệt gọi là phương thức khung trong một lớp gọi là lớp khung (Template class) và cài đặt các phần còn lại của thuật toán trong những lớp con của lớp này.

| 8.10.2 CẤU TRÚC MẪU TEMPLATE METHOD

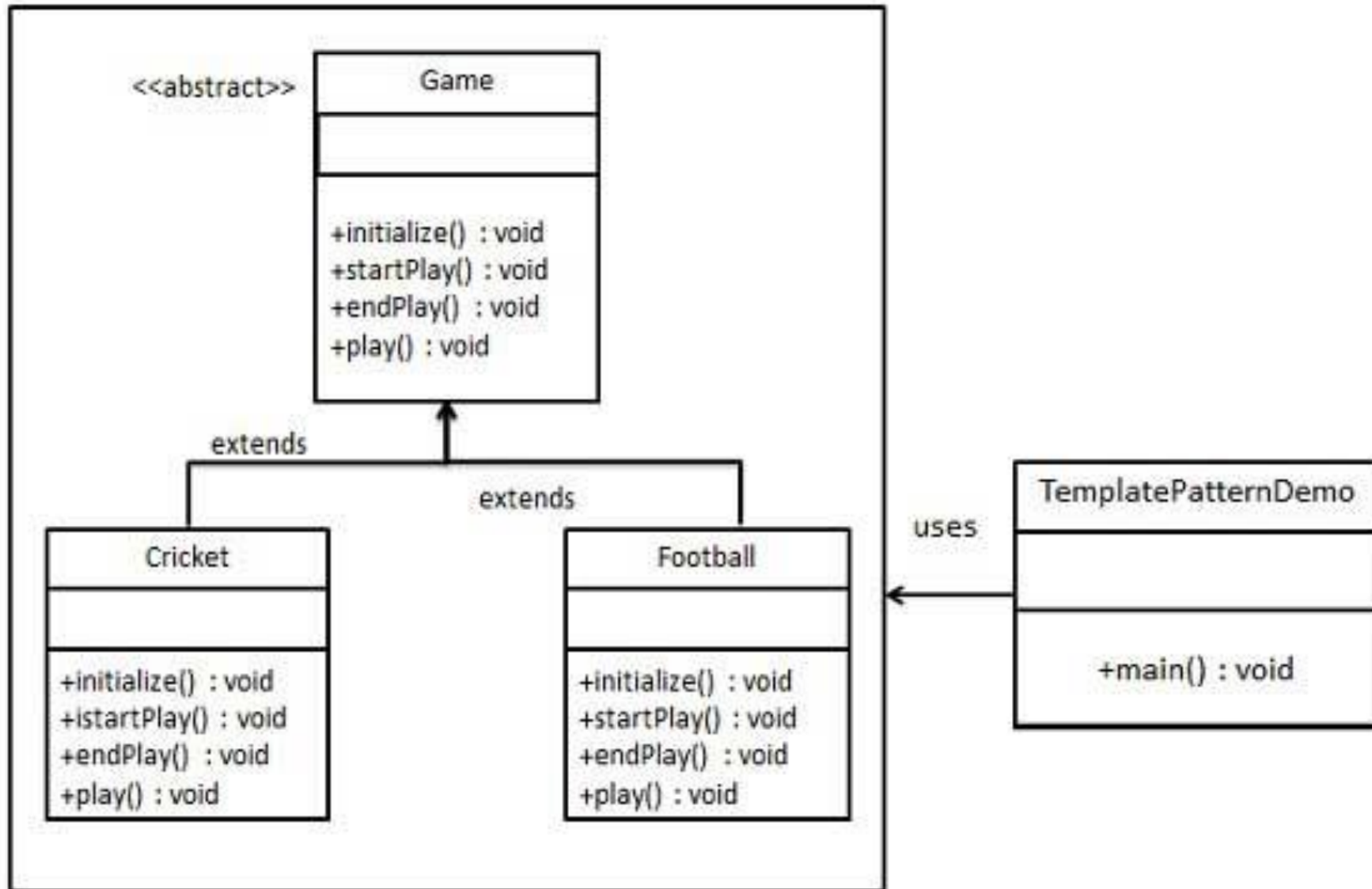


- **AbstractClass**: Định nghĩa các phương thức trừu tượng cho từng bước có thể được điều chỉnh bởi các lớp con. Cài đặt một phương thức duy nhất điều khiển thuật toán và gọi các bước riêng lẻ đã được cài đặt ở các lớp con.
- **ConcreteClass**: là một thuật toán cụ thể, cài đặt các phương thức của **AbstractClass**. Các thuật toán này ghi đè lên các phương thức trừu tượng để cung cấp các triển khai thực sự. Nó không thể ghi đè phương thức duy nhất đã được cài đặt ở **AbstractClass** (`templateMethod`).

| 8.10.3 TÌNH HUỐNG ÁP DỤNG TEMPLATE METHOD

- Khi có một thuật toán với nhiều bước và mong muốn cho phép tùy chỉnh chúng trong lớp con.
- Mong muốn chỉ có một triển khai phương thức trừu tượng duy nhất của một thuật toán.
- Mong muốn hành vi chung giữa các lớp con nên được đặt ở một lớp chung.
- Các lớp cha có thể gọi các hành vi trong các lớp con của chúng một cách thống nhất.

| 8.10.4 VÍ DỤ TEMPLATE METHOD



| 8.10.4 VÍ DỤ TEMPLATE METHOD

Bước 1: Tạo lớp trừu tượng.

```
public abstract class Game {  
    abstract void initialize();  
    abstract void startPlay();  
    abstract void endPlay();  
    //template method  
    public final void play(){  
        //initialize the game  
        initialize();  
        //start game  
        startPlay();  
        //end game  
        endPlay();  
    }  
}
```

Bước 2: Tạo lớp cụ thể

```
public class Cricket extends Game {  
    @Override  
    void endPlay() {  
        System.out.println("Cricket Game Finished!");  
    }  
    @Override  
    void initialize() {  
        System.out.println("Cricket Game Initialized! Start playing.");  
    }  
    @Override  
    void startPlay() {  
        System.out.println("Cricket Game Started. Enjoy the game!");  
    }  
}
```

| 8.10.4 VÍ DỤ TEMPLATE METHOD

Bước 2: Tạo lớp cụ thể

```
public class Football extends Game {  
    @Override  
    void endPlay() {  
        System.out.println("Football Game Finished!");  
    }  
    @Override  
    void initialize() {  
        System.out.println("Football Game Initialized! Start playing.");  
    }  
    @Override  
    void startPlay() {  
        System.out.println("Football Game Started. Enjoy the game!");  
    }  
}
```

| 8.10.4 VÍ DỤ TEMPLATE METHOD

Bước 3: Sử dụng khung Game play().

```
public class TemplatePatternDemo {  
    public static void main(String[] args) {  
  
        Game game = new Cricket();  
        game.play();  
        System.out.println();  
        game = new Football();  
        game.play();  
    }  
}
```

Bước 4: Xem kết quả

Cricket Game Initialized! Start playing.
Cricket Game Started. Enjoy the game!
Cricket Game Finished!

Football Game Initialized! Start playing.
Football Game Started. Enjoy the game!
Football Game Finished!

| Bài tập thực hành

Cài đặt thực hành ví dụ các mẫu trong bài học, bao gồm `client` và hàm `main()` để chạy chương trình.